Features

And what interpretability is actually for

The question we ended on

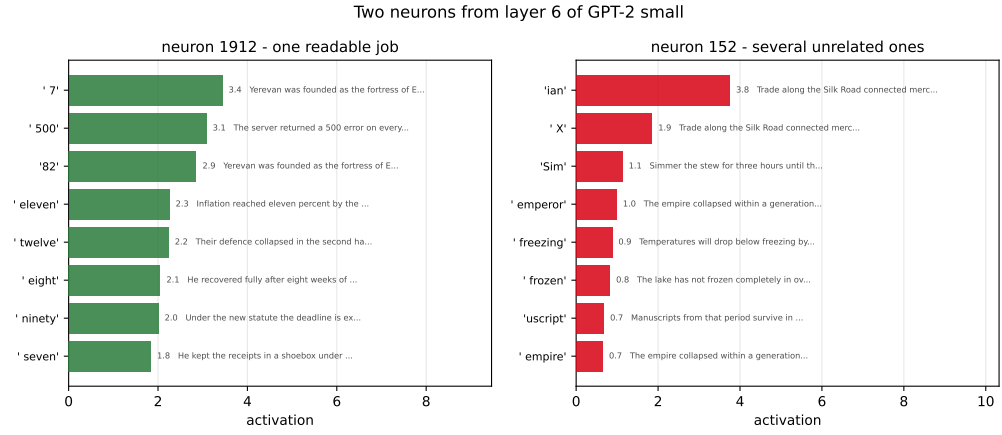
Two lectures of methods, all of them operating on **heads**, **layers** and **neurons** - the units the architecture happens to give us.

And the results kept being strange: a head that suppresses the right answer, four heads waiting to take over, a head that stares at the answer and writes nothing.

What if heads and neurons are simply the **wrong units**?

Compare: you can describe a chemical reaction atom by atom, and you will be correct and learn nothing. **Molecules** are the level at which chemistry becomes sayable.

Two neurons



Top activations over 66 sentences spanning ten unrelated domains. **Left:** numbers, every time, whatever the topic. **Right:** the Silk Road, simmering a stew, Roman emperors, freezing weather.

One of those is the exception

Neuron 1912 has **one readable job** - number words, in a finance sentence, a programming sentence and a history sentence alike. You could name it.

Neuron 152 has **several**, unrelated. You cannot name it, because there is no single thing to name.

Polysemanticity. One neuron, many unrelated meanings - and in a real network this is the **normal case**. "Find the neuron for X" is usually the wrong question, and every method from the last two lectures inherits the problem.

Remember lecture 1: top-activating examples are exactly the method that told a clean, convincing, *wrong* story about a BERT neuron. This is weak evidence on purpose - it is the evidence these units can give.

Outline

Superposition

Why the network does this to us on purpose

Predict first

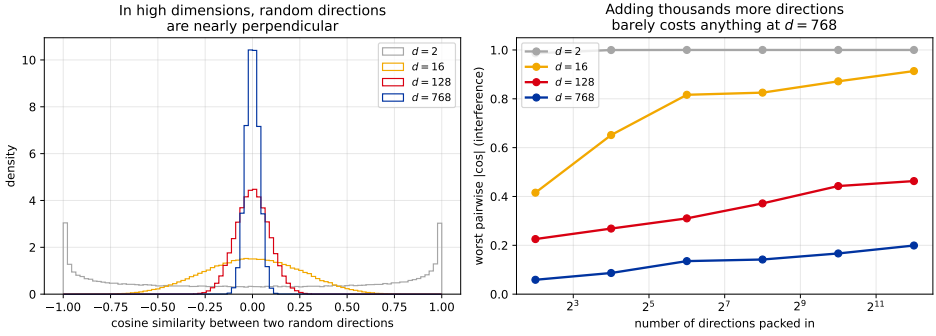
GPT-2 small's residual stream is **768 numbers** wide.

How many distinct concepts can it represent?

fewer than 768 exactly 768 many more than 768

The intuitive answer is 768 - one concept per dimension, like columns in a dataframe.

The intuition is wrong, and the geometry says why



Left: in two dimensions two random directions are usually far from perpendicular; by 768 the distribution collapses onto zero. **Right:** pack in more directions and interference grows - but slowly.

The numbers behind the right panel

	4 directions	**4096 directions**	
$d = 16$	0.42	0.91	unusable
$d = 128$	0.23	0.46	workable
$d = 768$	**0.06**	**0.20**	barely noticed

Read the bottom row across: multiply the number of stored directions by **a thousand** and the worst interference goes from 0.06 to 0.20.

At width 768 you can hold **4096** directions - more than five times the number of dimensions - with no pair overlapping by more than **0.20**. Nearly-orthogonal is cheap. **Exactly**-orthogonal is what costs a dimension.

So the network takes the deal

Two facts about the concepts a language model needs:

- there are **far more** of them than it has dimensions;
- almost all of them are **off** at any given moment - a sentence about cheese does not need the feature for Armenian grammar or for Python syntax.

Superposition. Given sparse features and limited width, the network stores more features than it has dimensions, accepting a little interference in exchange for a lot of capacity. Polysemantic neurons are the **symptom**; this trade is the cause.

The model is not being difficult. It is being efficient, and our preferred unit of analysis is the casualty.

Elhage et al. (2022), *Toy Models of Superposition*.

The assumption underneath all of this

Everything we are about to do rests on one idea:

The linear representation hypothesis

Concepts are stored as **directions**, and combinations of concepts are **sums** of those directions.

It has good evidence: steering by adding a vector works, probes find things, the whole attribution arithmetic from lecture 1 relies on it.

It is a **hypothesis**, not a theorem. There is active work arguing that important structure is not linear at all. When you read that a feature "is" a direction, the word doing the work is an assumption.

Sparse autoencoders

You have already built one of these

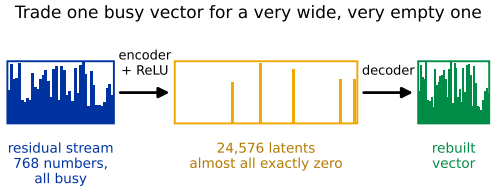
Back to the autoencoder chapter

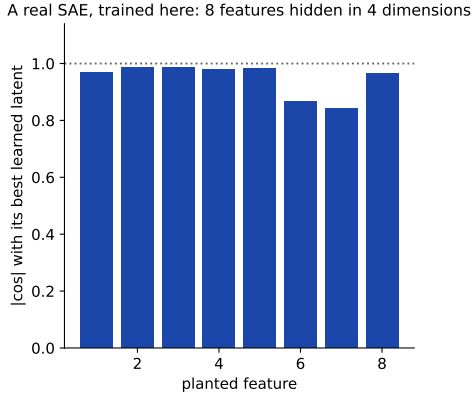
In chapter 8 you built a sparse autoencoder and ran it on a small recurrent network's hidden state. You already know:

- encoder $\rightarrow$ a **wide, sparse** code $\rightarrow$ decoder;
- the decoder columns form a **dictionary** of directions;
- the sparsity penalty is L1, the same one from Lasso in chapter 1;
- L_0 - how many features fire at once - is the number that matters.

Nothing about the method is new today. What is new is the **scale**, and the fact that you will **use** one rather than train it.

What it does, and that it works





Every planted feature recovered (worst 0.84). Mean 3.6 of 32 latents active per input.

Right: we planted 8 features in 4 dimensions - guaranteed superposition, since there are more features than axes - and trained a small dictionary on it. Every planted feature came back out.

Why sparsity is the whole trick

Nothing in that training told the dictionary what a feature *is*. There are no labels. The only pressure is: **rebuild the input, and keep almost everything off**.

If a direction has to explain many inputs at once it stays dense, and the penalty punishes it. The cheapest way to satisfy both terms is to give each *recurring, separable* thing its own direction. Sparsity is what turns "compress this" into "find the parts".

Which also says exactly when it fails: a concept that is always on, or one that never appears alone, has no reason to get its own direction. The method finds what is **sparse and separable** - not what is *important*.

What changes at language-model scale

	Your homework	**A real one**
Read from	a 32-unit hidden state	a 768- to 5000-wide residual stream
Dictionary size	512 features	tens of thousands to millions
Ground truth	you generated the corpus	none whatsoever
Who trains it	you, in a notebook	a lab, over weeks

The third row is the hard one. In your homework you could check a feature's name because you built the data. On real text, **nobody can**.

Which is why the discipline from lecture 2 carries over unchanged: naming a feature is a guess, and **ablating it is the test**.

Worked: how wide does the dictionary have to be?

Suppose a model tracks **10,000** concepts and typically has **20** of them active at once. The residual stream is **768** wide.

concepts to separate	10,000	we want one dictionary column each
dimensions available	768	so ∼13 concepts share every dimension
active at once	20	$L_0 = 20$ out of 10,000
sparsity	**0.2%**	of the dictionary fires on any one token

The dictionary is **13 times wider** than the space it reads from, and almost all of it is off at any moment. That is the whole design: **overcomplete and sparse**.

Which is why L_0 is the number people quote. It is the honest measure of whether a dictionary is actually separating concepts or just spreading them out.

You do not have to train one

Gemma Scope 2 (Google DeepMind, December 2025) is an open suite of sparse autoencoders and transcoders covering **every layer** of every Gemma 3 model, from 270 million to 27 billion parameters, pretrained and instruction-tuned.

Producing it took roughly **110 petabytes** of stored activations and over a trillion trained parameters.

> That work is done, and it is free. The interesting question stopped being "can we train a dictionary" and became **"what do we ask it?"**

Neuronpedia hosts the results: a browsable index of features with their top activations, their effects on the output, and their connections. No installation, no GPU, no download.

Let us actually look

Live: `neuronpedia.org` - pick a model, search for a concept, read the feature.

What to look at while we browse:

- the **top activating** text for a feature - is it one thing?
- what the feature **does to the output** when it fires;
- how many features look like near-duplicates of each other.

Keep lecture 1 in mind while it looks impressive. We are reading top-activating examples again - the method that has already fooled this field once.

Where sparse autoencoders are weak

- **Feature splitting.** Widen the dictionary and one broad feature becomes several narrow ones. So the "true" number of features is not well defined - it is a function of how big a dictionary you trained.
- **Dead features.** A large fraction never fire at all, wasting the dictionary.
- **No ground truth.** We cannot check the answers, only their plausibility.
- **They describe representation, not computation.** A list of what is present says nothing about what is *done* with it - the lecture-1 problem, one level up.

A 2025 paper states it flatly in the title: *Sparse Autoencoders Do Not Find Canonical Units of Analysis*.

Current advice from the field, roughly: **use them as a tool inside a project about something else**, not as the project.

Attribution graphs

From what is represented to what is computed

The gap that needs closing

Lecture 2 gave us	circuits, by hand, over months, for one task
Sparse autoencoders give us	good units, and no circuit

What we want is lecture 2's **circuit**, drawn automatically, in lecture 3's **units**.

The obstacle is the multi-layer perceptron blocks: a sparse autoencoder describes the activations going in and coming out, but not the map between them.

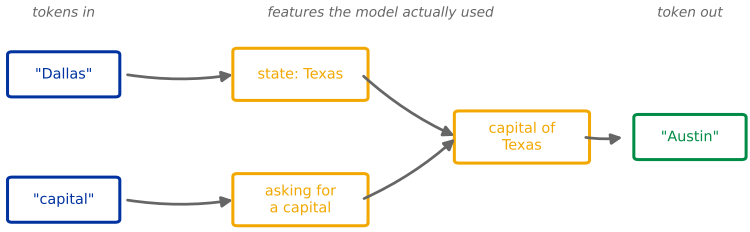
Transcoders

A **transcoder** replaces an entire block with a sparse, interpretable approximation of *what it computes* - input features in, output features out.

A sparse autoencoder makes the **state** readable. A transcoder makes the **step** readable. Chain the steps and you have a graph.

Attribution graphs

Trace which features cause which, all the way to the output, and you get a wiring diagram for one specific prompt:



This is lecture 2's project - done automatically, in minutes, in units that mean something. The single biggest practical change in this field in the last two years.

What that diagram is, and is not

- **Every arrow is a causal claim**, tested the way lecture 2 tested them: suppress a node and watch whether the ones downstream move.
- **It is specific to one prompt.** Change the wording and you get a different graph. This is not a diagram of the model.
- **Real graphs are not this tidy** - hundreds of nodes, and the readable version is a pruned view of a much larger object.

You can generate one today without training anything: `circuit-tracer` is open source (Gemma-2-2B, Llama-3.2-1B, Qwen3-4B) with a browser front end on Neuronpedia.

After Ameisen, Lindsey, Pearce et al. (2025), *Circuit Tracing*.

Before the findings: what kind of evidence is this?

The next two slides report results from that work. Since this lecture ends by asking you to demand exactly this, it is worth stating up front:

Not	somebody read a chain of thought and formed an impression
Not	top-activating examples with a story attached
But	features identified, then **suppressed and substituted**, with the output measured

In other words: the same standard as lecture 2, applied to features instead of heads. **They intervened.** That is why these results are worth repeating to you.

It is still one model family, and replication elsewhere is thin. Two things can be true: the method is sound and the coverage is narrow.

Predict first

Ask a model to write a rhyming couplet. It produces the second line one token at a time, left to right, because that is what next-token prediction means.

When does it decide what the last word will be?

When it gets there - it steers into a rhyme at the end.

Before writing any of the line.

The first answer is what the training objective suggests. Commit before the next slide.

What they found: the model plans ahead

Ask a model to write a rhyming couplet. The natural assumption is that it writes the second line word by word and arranges for a rhyme when it arrives at the end.

The graph shows otherwise. The feature for the **rhyming word at the end of the line** is active *before the model has written the words leading up to it*. It picks the destination, then writes a line that gets there.

Next-token prediction is the **training objective**. It is not a description of what the model does internally. Those are different claims, and this is the clearest evidence we have that they come apart.

Lindsey et al. (2025), *On the Biology of a Large Language Model*.

And: concepts are shared across languages

The same features fire for a concept whether the prompt is in English, French or Chinese. The model appears to translate into an internal representation, operate there, and translate back out.

> For a bilingual room this is worth sitting with: the evidence suggests the model is not running a separate Armenian model and English model. It has **one** set of concepts, with language as something closer to an input and output format.

A caution: "appears to" is doing real work in that sentence. The finding is about feature overlap in one model family, not a general law about multilingual networks.

Try it yourself

Live: `neuronpedia.org/graph` - type a prompt, get its attribution graph.

The thing to notice: this took a research team **months** in lecture 2, on a smaller model, for a simpler task. It now takes a text box and about a minute.

That is what progress looks like in this field - not a better theory, but a tool that makes the old work cheap.

Steering

If a concept is a direction, push in that direction

The simplest possible intervention

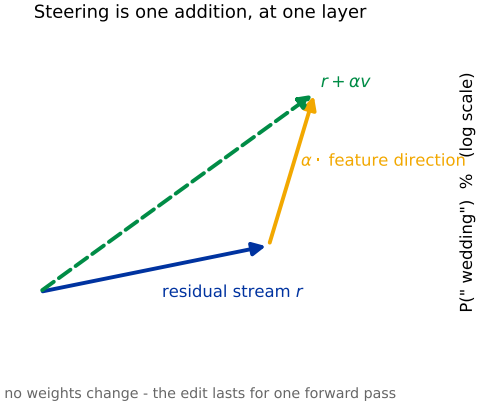
If the linear representation hypothesis holds, then to make a model more about something, you add that something's direction to the residual stream:

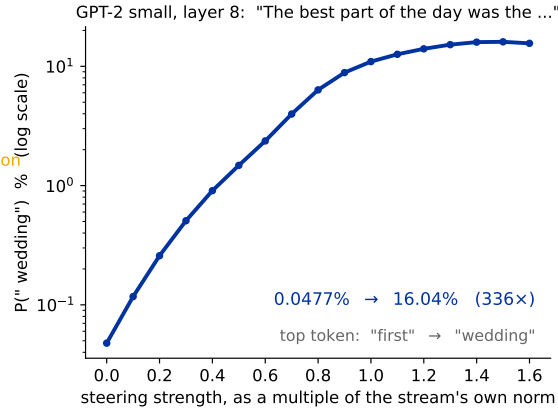
$$\text{activation} \leftarrow \text{activation} + \alpha \cdot \mathbf{v}_{\text{concept}}$$

You have seen this move before. In the diffusion chapter, classifier-free guidance pushed a generation toward a prompt by moving along a direction. Same idea, different object.

It is the cheapest control mechanism there is: no fine-tuning, no retraining, one vector addition at inference time.

What that looks like, and what α has to be





The direction is just the average residual stream on wedding sentences minus the average on matched neutral ones - a difference of means, nothing cleverer. At full strength the model's **top prediction changes from “first” to “wedding”**.

The detail that catches everyone

α is meaningless on its own. It only means something **relative to the vector you are adding it to**.

At this layer the residual stream on our prompt has norm **119**. Add a unit vector to it and you have changed it by under one percent - the output does not move, and it looks like the method failed.

So steering strengths are quoted as a **multiple of the activation's own norm**, which is how the axis on the previous slide is drawn. The same α at a different layer is a different intervention.

It caught the first version of that experiment: α from 0 to 8, against a stream of norm 119, moved nothing - which reads like evidence that steering fails.

Golden Gate Claude

You met this in chapter 8. Anthropic found the feature for the Golden Gate Bridge and clamped it on.

The model brought the bridge into everything - recipes, arithmetic, its own description of itself. Asked what it was, it said it was the bridge.

Why it matters as **evidence**, not as a joke: it demonstrates the feature was **causal**. Naming a feature from its top activations is a guess. Clamping it and watching identity-level behaviour follow is a test - the lecture-2 standard, applied to a feature.

The refusal direction

A model's willingness to refuse a request turns out to be mediated substantially by a **single direction** in activation space.

Add it

The model refuses more, including things it should answer.

Subtract it

Safety training stops applying. This is a jailbreak, and it needs no prompt engineering at all.

The same result is a safety tool and an attack. Interpretability is not inherently defensive - understanding a mechanism is exactly what you need in order to disable it.

This is a good argument for open-weight interpretability research being done in public, and a good argument against it. Both are real.

Steering is not editing

Steering vectors are **brittle**. The effect often does not generalise past the prompts it was built on, it degrades the model's general capability, and it has off-target consequences nobody looked for.

A steering vector is a demonstration that a direction is causal. It is not a reliable way to change what a model does in production.

Useful rule: treat steering as **evidence about the model**, not as a **feature of your product**.

Does the model mean what it says?

The part of this chapter you will use tomorrow

You all use these

Modern reasoning models print their chain of thought - a paragraph of working before the answer. You read it, and it is genuinely useful.

Is that text a **transcript** of the computation,
or a **story about** the computation?

Answer honestly before the next slide. Almost everyone reads it as a transcript - that is what it looks like.

Faithfulness, and how to test it

Faithfulness is the technical name for that assumption: does the stated reasoning correspond to the computation that produced the answer?

It is testable, and the shape of the test is the one from lecture 2:

1. change something that provably affects the answer;
2. check whether the stated reasoning mentions it.

Where the answer moves and the explanation does not, the text is **not** a record of the computation. And this is measurable - it is not a philosophical worry.

Two black-box tests worth knowing

Neither of these needs weights, hooks, or a GPU. Both belong in the same toolkit.

- **Token forcing.** Do not ask the model to answer - *begin* its answer for it, then let it continue. A model that refuses when asked plainly will often carry on happily from a first sentence it did not choose.
- **Prefill attacks.** The same move used adversarially: put words in the assistant's mouth to get past a behaviour that only ever lived in the opening tokens.

> What they reveal is where a behaviour is **implemented**. If a refusal survives having its first sentence written for it, it is doing real work. If it evaporates, it was a habit about how to start talking.

Cheap, fast, and a sensible first move before reaching for anything in the last two lectures.

What the graphs show

The attribution-graph work found cases where the model's **stated reasoning does not match its traced computation** - the written steps and the causal path diverge.

> We are not inferring this from behaviour. We can see one path in the graph and read a different one in the text.

Two consequences, and the second is the serious one:

- a plausible explanation is not evidence the answer is right;
- **reading the chain of thought is only oversight to the degree it is faithful.** If a model can reach an answer by a route its transcript never mentions, watching the transcript is not watching the model.

What this means for you, concretely

The reasoning trace is **evidence about** the answer. It is not a **proof of** the answer.

Do not treat "the model explained its steps" as verification. That is the same category error we made in lecture 1, at full scale: a probe finding information is not the model using it, and a model narrating a method is not the model having used it.

And the honest limit: we can detect **some** unfaithfulness. We cannot certify faithfulness. No procedure today takes a chain of thought and returns "verified".

Interpretability for reasoning models is one of the field's most active gaps - the models people actually use are the ones we understand least.

What this is for

Sorting the parts you can use from the parts you watch

Probes, in production

Of everything in these three lectures, this is the technique most likely to end up in your job.

- A probe is logistic regression on activations - lecture 1, chapter 3.
- Cheap to train, cheap to run, runs alongside the model.
- Already deployed for real content monitoring.

If you are running an open-weight model and you need to know whether it is about to do something - a probe on its activations is a better detector than a classifier on its output, because it fires **before** the text exists.

With lecture 1's caveat attached forever: a probe tells you the information is **present**. Validate against behaviour before you trust it as a control.

Debugging a model you actually ship

The canonical example is embarrassing and real: models insisting that **9.8 is smaller than 9.11**.

Not a knowledge gap - they know which is bigger if you ask differently. Interpretability work traced the internal features responsible, including ones that treat the strings as version numbers and dates.

The general move: **when a model fails strangely and the input looks fine, the explanation is in the activations.** Behavioural testing tells you *that* it failed. Nothing else tells you *why*.

Reading a model before you trust it

Most of you will deploy **open-weight** models - Gemma, Llama, Qwen - which are exactly the models the tooling in this lecture supports.

- Neuronpedia and `circuit-tracer` need no training and no GPU.
- Gemma Scope 2 covers every layer of every Gemma 3 size.

Frame it as **due diligence**, not research. Before you put a model in front of customers, you can now ask what it is doing on your actual inputs - and that is a new option, about two years old.

Evaluating claims made at you

You will be told that a model "understands" your domain, "reasons about" your data, or "knows" when it is uncertain. Sometimes by a vendor, sometimes by a paper, sometimes by a colleague.

Did anyone intervene, or did they just look?

That one question separates the two halves of this chapter, and you can ask it without running anything. It is the most portable thing here.

What the labs are doing with it

Compressed deliberately - this is the part you **watch**, not the part you use.

- **Auditing games.** Hide a goal inside a model, then see whether a red team can find it with interpretability tools. Worth knowing about because it manufactures the **ground truth** this field otherwise lacks.
- **Model organisms.** Deliberately install a known behaviour so methods can be tested against a known answer. Same motivation.
- **Monitoring reasoning at scale**, which is section 5's problem with a budget.

MIT Technology Review named mechanistic interpretability one of its ten breakthrough technologies of 2026. Context for why the field suddenly has money - **not** an argument that it works.

The asymmetry, said out loud

Most of the headline results in this chapter come from three or four well-funded labs, with model access, compute and staff that nobody in this room has.

That is a real limitation on the field, and you should factor it into how you read the literature.

It is **not** a reason the techniques are useless. Probes, patching, attribution and ablation all work fine on a 124-million-parameter model on a laptop - as every figure in these three lectures demonstrates. All of them were made on this machine.

Where this leaves us

An honest summary, and how to go further

What is genuinely unsolved

- **No ground truth.** We usually cannot check whether an explanation is right, which is why auditing games and model organisms exist at all.
- **No canonical units.** Neurons are polysemantic; sparse-autoencoder features split as you widen the dictionary. The right unit is an open question.
- **Coverage.** We explain individual behaviours on individual prompts. Nobody explains a model.
- **Reasoning models** - the ones everyone uses - are the least understood.
- **It does not scale by hand.** Every result in these lectures took a human days. Getting models to do the interpreting is an active area, and nobody has made it reliable.

A good map of the open problems: Sharkey et al. (2025), *Open Problems in Mechanistic Interpretability*. The foundational read is Elhage et al. (2021), *A Mathematical Framework for Transformer Circuits* - where the residual-stream picture from lecture 1 comes from.

The sentence to leave with

We can now say useful, testable things about what is inside a language model.

We cannot yet explain one.

Both halves are true, and people will try to sell you either half on its own.

If you want to actually do this

Start here	**ARENA**, chapter 1 - exercises with solutions. Sections 1.2 (the library), 1.4.1 (the circuit from lecture 2), 1.3.3 (features).
Zero setup	`neuronpedia.org` - browse features and attribution graphs today.
Libraries	**TransformerLens** for models up to about 9B - what these lectures used. **nnsight** for larger models.
The guide	Neel Nanda, *How To Become A Mechanistic Interpretability Researcher* - opinionated, current, and free.
Programmes	MATS, SPAR, MARS - structured mentorship, applications a few times a year.

On *project* choice: we taught lecture 2's circuit because it is the clearest worked example, not because it is where the field is going. Nanda's guide lists hand-traced circuits and toy tasks like grokking among the **fading** directions.

The chapter, in one table

	What we could do	What it could not tell us
Lecture 1	read the internals: logit lens, probes, attention, naming	whether any of it *mattered*
Lecture 2	test it: ablation, patching, attribution, a full circuit	whether heads were the right unit
Lecture 3	better units: features, transcoders, attribution graphs	whether the explanation is *correct*

Each lecture solved the previous one's complaint and produced a new one - an accurate picture of a field about eight years old.

What survives all three: **a claim about a model's internals is worth what its intervention is worth.**